

There are seven common orders of big O notation: $O(1)$, or constant time, describes code that doesn't change as the size of the data n grows; $O(\log n)$, or logarithmic time, describes code that increases by one step as the n data doubles in size; $O(n)$, or linear time, describes code that slows in proportion to the n data's growth; $O(n \log n)$, or n-log-n time, describes code that is a bit slower than $O(n)$, and many sorting algorithms have this order.

The higher orders are slower, because their runtime grows much faster than the size of their input data: $O(n^2)$, or polynomial time, describes code whose runtime increases by the square of the n input; $O(2^n)$, or exponential time, and $O(n!)$, or factorial time, orders are uncommon, but come up when combinations or permutations are involved, respectively.

Keep in mind that although big O is a helpful analysis tool, it isn't a substitute for running your code under a profiler to find out where any bottlenecks are. But an awareness of big O notation and how code slows as data grows can help you avoid writing code that is orders slower than it needs to be.